

# Design and Implementation of a 5-Stage Pipelined RISC-V Processor

With Data Forwarding, Hazard Mitigation,  
and Dynamic Branch Prediction

---

The RISCY Business

Saumya Vira (2024102044)  
Shrenil Patel (2024102066)  
Het Selarka (2024102031)

March 10, 2026

## Abstract

This report presents the design and Verilog implementation of a 64-bit, 5-stage pipelined processor based on the RISC-V ISA (RV64I subset). The processor partitions instruction execution into five stages—Instruction Fetch, Instruction Decode, Execute, Memory Access, and Write-Back—connected through pipeline registers. Three complementary hazard-resolution mechanisms are employed: (i) a combinational forwarding unit that eliminates stalls for most read-after-write data dependencies by multiplexing results from later pipeline stages back to the ALU inputs, (ii) a hazard detection unit that inserts exactly one pipeline bubble when a load instruction is immediately followed by a consumer, and (iii) a 2-bit saturating-counter branch predictor backed by a 64-entry Branch Target Buffer, which reduces the misprediction penalty to a single cycle by resolving branches in the Decode stage. A dedicated memory-to-memory forwarding path further eliminates stalls for store-after-load sequences. The design passes a comprehensive verification suite of 26 directed test scenarios covering every forwarding path, stall condition, and control-flow edge case.

## Contents

|          |                                           |          |
|----------|-------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                       | <b>4</b> |
| 1.1      | Supported Instruction Set . . . . .       | 4        |
| <b>2</b> | <b>Processor Microarchitecture</b>        | <b>4</b> |
| 2.1      | Top-Level Pipeline Organization . . . . . | 4        |
| 2.2      | Stage-by-Stage Description . . . . .      | 5        |
| 2.2.1    | Instruction Fetch (IF) . . . . .          | 5        |
| 2.2.2    | Instruction Decode (ID) . . . . .         | 5        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 2.2.3    | Execute (EX) . . . . .                                              | 5         |
| 2.2.4    | Memory Access (MEM) . . . . .                                       | 5         |
| 2.2.5    | Write-Back (WB) . . . . .                                           | 6         |
| 2.3      | Pipeline Register Specifications . . . . .                          | 6         |
| 2.3.1    | IF/ID Pipeline Register . . . . .                                   | 6         |
| 2.3.2    | ID/EX Pipeline Register . . . . .                                   | 6         |
| 2.3.3    | EX/MEM Pipeline Register . . . . .                                  | 6         |
| 2.3.4    | MEM/WB Pipeline Register . . . . .                                  | 7         |
| 2.4      | Pipeline Register Behavior Under Hazards . . . . .                  | 7         |
| <b>3</b> | <b>Data Forwarding Unit</b>                                         | <b>7</b>  |
| 3.1      | Forwarding Cases . . . . .                                          | 8         |
| 3.2      | EX Hazard Pseudocode . . . . .                                      | 8         |
| 3.3      | MEM Hazard Pseudocode . . . . .                                     | 8         |
| 3.4      | Priority and Double-Hazard Resolution . . . . .                     | 9         |
| 3.5      | Register File Write-First Bypass (WB → ID) . . . . .                | 9         |
| <b>4</b> | <b>Hazard Detection and Stall Logic</b>                             | <b>10</b> |
| 4.1      | The Load-Use Problem . . . . .                                      | 10        |
| 4.2      | Detection Conditions . . . . .                                      | 11        |
| 4.3      | Stall Actions (Three Simultaneous Effects) . . . . .                | 11        |
| 4.4      | Flush Logic (Control Hazards) . . . . .                             | 11        |
| 4.5      | Optimized Stall Condition . . . . .                                 | 12        |
| 4.6      | MEM-to-MEM Forwarding: Eliminating the Store-After-Load Stall . . . | 12        |
| 4.6.1    | Load-Store Forwarding Unit . . . . .                                | 12        |
| <b>5</b> | <b>Branch Prediction and Control Hazard Resolution</b>              | <b>13</b> |
| 5.1      | Design Rationale . . . . .                                          | 13        |
| 5.2      | 2-Bit Saturating Counter Predictor . . . . .                        | 13        |
| 5.3      | Branch Target Buffer . . . . .                                      | 13        |
| 5.4      | Misprediction Detection and Recovery . . . . .                      | 14        |
| <b>6</b> | <b>Comprehensive Hazard Taxonomy</b>                                | <b>14</b> |
| <b>7</b> | <b>Verification and Testing</b>                                     | <b>15</b> |
| 7.1      | Test Methodology . . . . .                                          | 15        |
| 7.2      | Directed Test Plan . . . . .                                        | 16        |
| 7.3      | Sample Test Walkthroughs . . . . .                                  | 16        |
| 7.3.1    | Test 1: EX-Stage Forwarding (ForwardA = 10) . . . . .               | 16        |
| 7.3.2    | Test 7: Load-Use Hazard (Stall + Forward) . . . . .                 | 17        |
| 7.3.3    | Test 10: Store After Load (MEM-to-MEM Forwarding) . . . . .         | 17        |
| 7.3.4    | Test 12: Branch Taken (Misprediction Flush) . . . . .               | 18        |
| <b>8</b> | <b>Simulation Results</b>                                           | <b>18</b> |
| 8.1      | Forwarding Waveform Analysis . . . . .                              | 18        |
| 8.2      | Stall Waveform Analysis . . . . .                                   | 18        |
| 8.3      | Flush Waveform Analysis . . . . .                                   | 18        |
| 8.4      | Stall Followed by Flush . . . . .                                   | 19        |

|           |                       |           |
|-----------|-----------------------|-----------|
| <b>9</b>  | <b>Module Summary</b> | <b>19</b> |
| <b>10</b> | <b>Conclusion</b>     | <b>19</b> |
| <b>11</b> | <b>Contribution</b>   | <b>20</b> |

# 1 Introduction

Pipelining is the principal micro-architectural technique for increasing instruction throughput in modern processors. Rather than executing one instruction from start to finish before beginning the next, a pipelined architecture overlaps the execution of consecutive instructions. Under ideal conditions a  $k$ -stage pipeline can sustain a throughput of one instruction per clock cycle, improving performance by a factor approaching  $k$  relative to an unpipelined implementation.

Real programs, however, exhibit *hazards*—situations in which the next instruction cannot execute in the immediately following clock cycle. Hazards fall into three categories:

1. **Data hazards** arise when an instruction references an operand that has not yet been written back by a preceding instruction.
2. **Control hazards** occur when the outcome of a branch instruction is not known before the next fetch must be initiated.
3. **Structural hazards** appear when two pipeline stages simultaneously require the same hardware resource (eliminated here by using separate instruction and data memories).

## 1.1 Supported Instruction Set

Table 1: Encoded instruction types supported by the processor.

| Format | Operations                                          | Opcode [6:0] | Semantics                           |
|--------|-----------------------------------------------------|--------------|-------------------------------------|
| R-type | ADD, SUB, AND, OR, XOR,<br>SLT, SLTU, SLL, SRL, SRA | 0110011      | $rd \leftarrow rs1 \text{ op } rs2$ |
| I-type | ADDI                                                | 0010011      | $rd \leftarrow rs1 + imm$           |
| Load   | LD                                                  | 0000011      | $rd \leftarrow M[rs1 + imm]$        |
| Store  | SD                                                  | 0100011      | $M[rs1 + imm] \leftarrow rs2$       |
| Branch | BEQ                                                 | 1100011      | if ( $rs1 == rs2$ ) PC += imm       |

# 2 Processor Microarchitecture

## 2.1 Top-Level Pipeline Organization

Figure 1 shows the five stages and the four pipeline registers that separate them. Each clock edge latches a new snapshot of datapath and control signals from one stage into the next register, allowing up to five instructions to be in flight simultaneously.

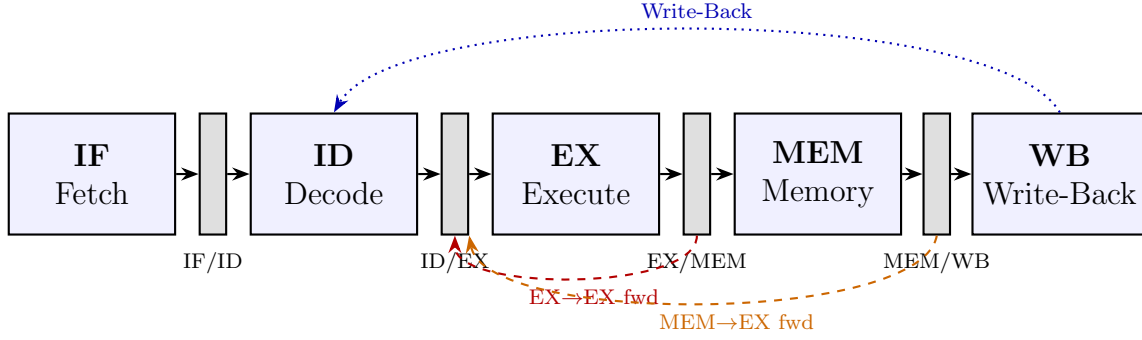


Figure 1: Five-stage pipeline with forwarding paths and write-back.

## 2.2 Stage-by-Stage Description

### 2.2.1 Instruction Fetch (IF)

The Program Counter (PC) indexes into a byte-addressable instruction memory (4096 B) to retrieve a 32-bit instruction. The branch predictor simultaneously examines the current PC and produces a prediction, selecting  $PC_{next}$  from three candidates:

$$PC_{next} = \begin{cases} \text{correct\_pc} & \text{if misprediction (from ID),} \\ \text{predict\_target} & \text{if predict\_taken,} \\ PC + 4 & \text{otherwise (sequential).} \end{cases}$$

On a stall, both the PC register and the IF/ID pipeline register hold their values.

### 2.2.2 Instruction Decode (ID)

The 32-bit instruction word is decomposed into opcode, `rs1`, `rs2`, `rd`, `funct3`, and `funct7[5]`. A combinational control unit generates seven control signals (`reg_write`, `alu_src`, `mem_read`, `mem_write`, `mem_to_reg`, `branch`, `alu_op[1:0]`). The register file is read and the immediate is sign-extended.

Critically, **branch resolution is performed here** (not in EX), using a dedicated equality comparator on the register-file outputs. This limits the branch misprediction penalty to a single fetch cycle.

### 2.2.3 Execute (EX)

The 64-bit ALU operates on two inputs selected by forwarding muxes. The ALU control unit decodes `alu_op`, `funct3`, and `funct7[5]` into a 4-bit operation code supporting ten operations: ADD, SUB, AND, OR, XOR, SLT, SLTU, SLL, SRL, and SRA.

### 2.2.4 Memory Access (MEM)

A 1024-byte data memory performs load or store operations. A MEM-to-MEM forwarding mux can bypass the write-back value of a load directly to the store data port, avoiding an unnecessary stall for store-after-load sequences.

### 2.2.5 Write-Back (WB)

A final mux selects either the ALU result or the memory read data and writes it back to the register file.

## 2.3 Pipeline Register Specifications

### 2.3.1 IF/ID Pipeline Register

- **Inputs:** 32-bit `instr`, 64-bit `pc`, 1-bit `predicted_taken`
- **Outputs:** 32-bit `instr_IF_ID`, 64-bit `IF_ID_pc_out`, 1-bit `IF_ID_predicted_taken`

The IF/ID register stores the fetched instruction and the associated program counter. On a **stall**, both the PC and IF/ID retain their values so that the same instruction can be re-decoded in the next cycle. On a **flush** (branch misprediction), the register is cleared to produce a NOP.

### 2.3.2 ID/EX Pipeline Register

- **Inputs:**
  - 64-bit: `IF_ID_pc_out`, `read_data1`, `read_data2`, `imm`
  - 5-bit: `rs1`, `rs2`, `rd`
  - 3-bit: `funct3`
  - 1-bit: `funct7_bit5`
  - Control (via mux): `reg_write`, `alu_src`, `mem_read`, `mem_write`, `mem_to_reg`, `branch`
  - 2-bit: `alu_op`
- **Outputs:** Same signals prefixed with `ID_EX_`

This is the most critical pipeline register: it houses the control signals generated by the control unit. The inputs with the `_out_mux` suffix come from a multiplexer that selects between the actual control signals and zeros (for NOP insertion during stalls). The register also stores `rs1`, `rs2`, and `rd` for forward-checking in subsequent stages. Note that the ALU control output (not raw `alu_op`) is passed through, since ALU decoding occurs in the ID stage.

### 2.3.3 EX/MEM Pipeline Register

- **Inputs:**
  - 64-bit: `alu_result`, `write_data` (`read_data2` for stores)
  - 5-bit: `rs2_ID_EX`, `rd_ID_EX`
  - 1-bit: `mem_to_reg`, `reg_write`, `mem_read`, `mem_write`
- **Outputs:** Same signals suffixed with `EX_MEM`

The EX/MEM register stores the ALU result, the data to be stored (for `sd`), the destination register, and the `rs2` address (required for MEM-to-MEM forwarding comparisons).

### 2.3.4 MEM/WB Pipeline Register

- **Inputs:**

- 64-bit: `alu_result_EX_MEM`, `mem_read_data`
- 5-bit: `rd_EX_MEM`
- 1-bit: `mem_to_reg_EX_MEM`, `reg_write_EX_MEM`

- **Outputs:** Same signals prefixed with `MEM_WB`

The MEM/WB register stores the final ALU result and memory data to be selected by the write-back mux and written to the register file.

## 2.4 Pipeline Register Behavior Under Hazards

Table 2: Pipeline register actions for stall, flush, and normal operation.

| Register | Normal                        | Stall                 | Flush (Misprediction)           |
|----------|-------------------------------|-----------------------|---------------------------------|
| PC       | $\leftarrow PC_{\text{next}}$ | <i>Hold</i>           | $\leftarrow \text{correct\_pc}$ |
| IF/ID    | $\leftarrow$ IF outputs       | <i>Hold</i>           | <i>Clear to NOP</i>             |
| ID/EX    | $\leftarrow$ ID outputs       | <i>Clear (bubble)</i> | <i>Clear (bubble)</i>           |
| EX/MEM   | $\leftarrow$ EX outputs       | Normal                | Normal                          |
| MEM/WB   | $\leftarrow$ MEM outputs      | Normal                | Normal                          |

## 3 Data Forwarding Unit

The forwarding unit is a purely combinational block that detects register-name matches between the destination of an instruction in the MEM or WB stage and the source registers of the instruction currently in the EX stage. It produces two 2-bit mux-select signals, `ForwardA` (for `rs1`) and `ForwardB` (for `rs2`).

- **Inputs:** 5-bit `ID_EX.rs1`, `ID_EX.rs2`, `EX_MEM.rd`, `MEM_WB.rd`; 1-bit `EX_MEM.reg.write`, `MEM_WB.reg.write`
- **Outputs:** 2-bit `ForwardA`, `ForwardB`

The inputs to the ALU are selected by this forwarding unit. The forwarding logic and hazard conditions are detailed below.

Table 3: Forwarding mux encoding.

| Value | Source                         | Meaning                            |
|-------|--------------------------------|------------------------------------|
| 2'b00 | ID/EX register file output     | No hazard; use pipeline register   |
| 2'b10 | EX/MEM <code>alu_result</code> | 1-instruction-gap RAW (EX hazard)  |
| 2'b01 | MEM/WB write-back data         | 2-instruction-gap RAW (MEM hazard) |

### 3.1 Forwarding Cases

The cases where forwarding is required are:

1. **1-instruction gap (EX hazard):** Data used in an operation is changed in the previous instruction. The value is forwarded from the EX/MEM pipeline register to the EX stage:

```
add  x1, x2, x3      # produces x1
sub  x4, x1, x5      # ForwardA = 10 (from EX/MEM)
```

This scenario also holds for stores (`sd x1, 0(x4)`), branches (`beq x1, x3, 0x4`), and loads (`ld x4, 0(x1)`).

2. **2-instruction gap (MEM hazard):** Data used in an operation is changed two instructions earlier. The value is forwarded from the MEM/WB pipeline register:

```
add  x1, x2, x3      # produces x1
add  x4, x5, x6      # independent instruction
sub  x6, x1, x7      # ForwardA = 01 (from MEM/WB)
```

3. **Load followed by store (MEM-to-MEM):** When a load is followed by a store of the same register, the value is needed at the MEM stage (not EX), so we forward from MEM/WB directly to the data memory write port:

```
ld    x1, 0(x2)      # load x1 from memory
sd    x1, 0(x3)      # store x1 -- MEM-to-MEM forward
```

### 3.2 EX Hazard Pseudocode

The EX hazard check verifies whether the `rd` in EX/MEM matches `rs1/rs2` in ID/EX, while ensuring we are writing, and the destination is not `x0`.

```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0) and
    (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
    ForwardA = 10
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0) and
    (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
    ForwardB = 10
```

Listing 1: EX hazard condition (pseudocode).

### 3.3 MEM Hazard Pseudocode

The MEM hazard check additionally ensures that the EX hazard is emphnot already handling the same register (to correctly resolve double data hazards):

```
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd != 0) and
    not(EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0) and
        (EX/MEM.RegisterRd = ID/EX.RegisterRs1)) and
    (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
    ForwardA = 01
```

Listing 2: MEM hazard condition (pseudocode).



The same applies for ForwardB with RegisterRs2.

### 3.4 Priority and Double-Hazard Resolution

A *double data hazard* occurs when two consecutive instructions both write the same register and a third instruction reads it:

```

1 addi x12, x0, 1      # I    (MEM/WB stage)
2 addi x12, x12, 2     # I+1 (EX/MEM stage) -- newest value
3 add  x13, x12, x1     # I+2 (EX stage)      -- needs newest x12

```

Listing 3: Double data hazard: both I and I+1 write x12.

Both the EX and MEM forwarding conditions are true simultaneously. Correctness requires forwarding the *newest* value—the one in EX/MEM. The implementation evaluates EX hazard *after* MEM hazard so that its assignment takes priority. The MEM hazard condition also explicitly *excludes* the case where EX/MEM already supplies the same register:

```

1 always @(*) begin
2     ForwardA = 2'b00;
3     // MEM hazard -- excludes the case already handled by EX
4     // hazard
5     if (MEM_WB_reg_write && MEM_WB_rd != 5'd0
6         && !(EX_MEM_reg_write && EX_MEM_rd != 5'd0
7             && EX_MEM_rd == ID_EX_rs1)
8         && MEM_WB_rd == ID_EX_rs1)
9         ForwardA = 2'b01;
10    // EX hazard -- highest priority, overwrites MEM if both
11    // match
12    if (EX_MEM_reg_write && EX_MEM_rd != 5'd0
13        && EX_MEM_rd == ID_EX_rs1)
14        ForwardA = 2'b10;
15 end

```

Listing 4: ForwardA logic with double-hazard exclusion.

The same logic applies to ForwardB for the rs2 operand.

### 3.5 Register File Write-First Bypass (WB → ID)

A third forwarding path exists *inside* the register file module itself. When the WB stage writes a register in the same cycle that the ID stage reads it (a 3-instruction gap), a combinational bypass delivers the written data directly:

```

1 assign read_data1 = (rs1 == 5'd0) ? 64'd0 :
2                     (reg_write && rd != 5'd0 && rd == rs1) ?
3                     write_data : registers[rs1];

```

Listing 5: Write-first bypass in the register file.

This ensures correct behavior without requiring an additional forwarding mux in the main datapath.

## 4 Hazard Detection and Stall Logic

The hazard detection unit monitors dependencies between the instruction in the ID stage and the instruction in the EX stage. Its inputs and outputs are:

- **Inputs:** 5-bit IF\_ID\_rs1, IF\_ID\_rs2, ID\_EX\_rd; 1-bit ID\_EX\_mem\_read, IF\_ID\_mem\_write
- **Outputs:** 1-bit `stall` (also controls `pc_write`, `IF_ID_write`, and `control_mux_select`)

The types of cases that lead to stalls are described below.

### 4.1 The Load-Use Problem

Forwarding is sufficient to solve RAW data hazards when the result is computed in the EX stage, because its result can then be forwarded to the EX stage of the next instruction. Unfortunately, the `ld` instruction does not finish reading data until the end of the MEM stage, so its result cannot be forwarded to the EX stage of the next instruction. We thus say that the `ld` instruction has a **two-cycle latency** because a dependent instruction cannot use its result until two cycles later.

When a load instruction is followed by an instruction that uses the loaded value, we stall the pipeline by inserting a bubble. The following examples all trigger a load-use stall:

- **Load → ALU:**

```
ld    x1, 0(x2)
add   x3, x1, x4           # x1 not ready -- stall
```

- **Load → Branch:**

```
ld    x1, 0(x2)
beq   x1, x3, 0x4          # x1 not ready -- stall
```

- **Load → Load (address dependency):**

```
ld    x1, 0(x2)
ld    x3, 0(x1)           # x1 used for address -- stall
```

- **Load → Store (address dependency only):**

```
ld    x1, 0(x2)
sd    x4, 0(x1)           # x1 used for address -- stall
```

Note: if the store uses the loaded register as the *data* operand (`rs2`) rather than the address, MEM-to-MEM forwarding handles it without a stall.

| Cycle | IF  | ID  | EX            | MEM           | WB  | Note           |
|-------|-----|-----|---------------|---------------|-----|----------------|
| $N$   | add | ld  | ...           | ...           | ... |                |
| $N+1$ | add | ld  | <i>bubble</i> | ...           | ... | <b>Stall</b>   |
| $N+2$ | ... | add | ld            | <i>bubble</i> | ... | Resume; Fwd=01 |

Figure 2: Pipeline timing for a load-use hazard showing the inserted bubble.

## 4.2 Detection Conditions

```

1 wire rs1_hazard = ID_EX_mem_read && (ID_EX_rd != 5'd0)
2               && (ID_EX_rd == IF_ID_rs1);
3
4 wire rs2_hazard = ID_EX_mem_read && (ID_EX_rd != 5'd0)
5               && (ID_EX_rd == IF_ID_rs2)
6               && !IF_ID_mem_write;    // exclude store-after-
3               load
7
8 assign stall = rs1_hazard || rs2_hazard;

```

Listing 6: Hazard detection unit conditions.

The `rs2_hazard` condition explicitly **excludes stores**: when a store follows a load to the same register, the data is not needed at EX-time because the store's data port is consumed in the MEM stage, where MEM-to-MEM forwarding can supply it directly.

## 4.3 Stall Actions (Three Simultaneous Effects)

It is very important to hold the state of the registers and memories during a stall. The bubble is introduced by zeroing out the EX stage control signals during an ID stage stall so that the bubble performs no action and changes **no architectural state**.

The stall logic pseudocode is:

```

if (ID/EX.MemRead and
    ((ID/EX.RegisterRd = IF/ID.RegisterRs1) or
     (ID/EX.RegisterRd = IF/ID.RegisterRs2)))
    pc_write = 0;           // freeze PC
    IF_ID_write = 0;        // freeze IF/ID register
    control_mux_select = 1; // insert bubble (zero controls)
    flush = 0;              // no flush for stalls

```

Listing 7: Stall logic pseudocode.

The three simultaneous effects are:

1. **Freeze PC**: the program counter retains its current value.
2. **Freeze IF/ID**: the fetched instruction is preserved for re-decode next cycle.
3. **Bubble into ID/EX**: all control signals in the ID/EX register are zeroed, creating a NOP that propagates harmlessly through EX, MEM, and WB.

After the single-cycle stall, the loaded value reaches writing distance from MEM/WB and is forwarded via `ForwardA/B = 2'b01`.

## 4.4 Flush Logic (Control Hazards)

The flush mechanism handles branch mispredictions. When the branch outcome computed in ID disagrees with the prediction, the pipeline must discard the incorrectly fetched instruction in IF/ID. The flush logic asserts a `flush` signal that:

- Clears the IF/ID pipeline register to zero (inserting a NOP)

- Clears the ID/EX control signals to zero (inserting a bubble)
- Redirects the PC to the correct branch target or fall-through address

Note that for stalls, `flush` remains 0—the pipeline is merely paused, not squashed.

## 4.5 Optimized Stall Condition

The basic stall condition checks whether `ID_EX_rd` matches either `IF_ID_rs1` or `IF_ID_rs2` when a load is in the EX stage. However, this can be optimized for two special cases:

1. **Load followed by a load:** There is no `rs2` for a load, but the lower 5 bits of the immediate in the I-type encoding overlap with the `rs2` field. If `rs1` matches the `rd` of the previous load, it is a genuine stall regardless.
2. **Load followed by a store:** The `rs2` of the store is the data register, and its value is consumed in the MEM stage (not EX), so this dependency can be resolved by MEM-to-MEM forwarding. If `rs1` matches `rd`, a stall is needed anyway (for the address computation).

The updated stall condition excludes the store case from `rs2` matching by checking `!IF_ID_mem_write`, allowing the processor to skip the stall when MEM-to-MEM forwarding can handle the data dependency.

## 4.6 MEM-to-MEM Forwarding: Eliminating the Store-After-Load Stall

### 4.6.1 Load-Store Forwarding Unit

The load-store data hazard is handled by a dedicated forwarding path with the following interface:

- **Inputs:** 5-bit `MEM_WB_rd`, `EX_MEM_rs2`; 64-bit `EX_MEM_write_data`; 1-bit `MEM_WB_mem_to_reg`, `EX_MEM_mem_write`, `MEM_WB_reg_write`
- **Outputs:** 1-bit `mem_to_mem_forward` (mux select for data memory write port)

The select line controls a multiplexer that chooses between the MEM/WB register file write-back data and the data from the EX/MEM pipeline register.

A common idiom in generated code is:

```
1 ld  x1, 0(x5)      # Load x1
2 sd  x1, 16(x5)     # Store x1 -- value needed at MEM, not EX
```

Listing 8: Store-after-load pattern (no stall required).

Since the store consumes its data register only in MEM, the processor can forward the load’s result (emerging from MEM/WB) directly to the data memory’s write-data input in the same clock cycle:

```
1 wire mem_to_mem_forward =
2     EX_MEM_mem_write      &&    // current instruction is a store
3     MEM_WB_reg_write      &&    // preceding instruction writes a
    register
```

```

4     MEM_WB_mem_to_reg    &&    // preceding instruction was a load
5     (EX_MEM_rs2 == MEM_WB_rd) && // store's data reg matches
6     load's dest
7     (MEM_WB_rd != 5'd0);        // not x0
8 wire [63:0] MEM_store_data = mem_to_mem_forward ?
9                               WB_write_data : EX_MEM_write_data;

```

Listing 9: MEM-to-MEM forwarding condition.

This optimization completely *eliminates* the one-cycle stall penalty that a naïve hazard detector would insert, saving one cycle per store-after-load pair.

## 5 Branch Prediction and Control Hazard Resolution

### 5.1 Design Rationale

Branches are resolved in the **ID stage** using a dedicated equality comparator, rather than reusing the ALU in EX. This architectural choice limits the worst-case branch penalty to *one cycle* (the instruction already fetched in IF while the branch was in ID).

### 5.2 2-Bit Saturating Counter Predictor

A Branch History Table (BHT) with 64 entries stores a 2-bit saturating counter for each branch:

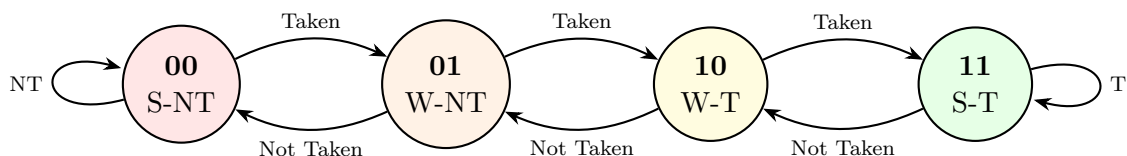


Figure 3: State-transition diagram for the 2-bit saturating counter. Entries are initialized to **01** (Weakly Not-Taken). A branch must be taken twice consecutively to reach the Strongly-Taken state, avoiding premature prediction flips for alternating patterns.

### 5.3 Branch Target Buffer

The BTB stores the target address of previously executed branches in 64 entries indexed by PC[7:2] (the two least-significant bits are always zero for 4-byte-aligned instructions):

```

1 wire [5:0] index_IF = pc_IF[7:2];
2 assign predict_taken = BHT[index_IF][1] & BTB_valid[index_IF];
3 assign predict_target = BTB_target[index_IF];

```

Listing 10: Prediction outputs from BHT and BTB.

A prediction of “taken” is issued only when both (a) the saturating counter’s MSB is 1, and (b) the BTB entry is marked valid.

## 5.4 Misprediction Detection and Recovery

At the end of the ID stage the actual branch outcome is compared with the stored prediction:

```

1 assign misprediction = ID_branch &
2                               (IF_ID_predicted_taken != branch_taken);
3 assign correct_pc = branch_taken ? branch_target : (IF_ID_PC +
4               64'd4);
5 assign flush = misprediction;
```

Listing 11: Misprediction detection in the ID stage.

On a misprediction, two things happen simultaneously:

- The IF/ID pipeline register is **flushed** (cleared to NOP), squashing the wrongly-fetched instruction.
- The PC is redirected to `correct_pc`.

Correctly-predicted branches—both taken and not-taken—incur **zero penalty cycles**.

## 6 Comprehensive Hazard Taxonomy

Table 4 summarizes every hazard scenario handled by the processor, the detection mechanism, the resolution strategy, and the associated cycle penalty.

Table 4: Complete catalog of pipeline hazards and their resolutions.

| Scenario                  | Gap                      | Resolution                       | Penalty |
|---------------------------|--------------------------|----------------------------------|---------|
| EX hazard on <b>rs1</b>   | $I \rightarrow I+1$      | ForwardA = (EX/MEM→EX)           | 10 0    |
| EX hazard on <b>rs2</b>   | $I \rightarrow I+1$      | ForwardB = (EX/MEM→EX)           | 10 0    |
| MEM hazard on <b>rs1</b>  | $I \rightarrow I+2$      | ForwardA = (MEM/WB→EX)           | 01 0    |
| MEM hazard on <b>rs2</b>  | $I \rightarrow I+2$      | ForwardB = (MEM/WB→EX)           | 01 0    |
| Double data hazard        | $I, I+1 \rightarrow I+2$ | EX hazard takes priority         | 0       |
| Load-use on <b>rs1</b>    | $LD \rightarrow I+1$     | 1-cycle stall + ForwardA=01      | 1       |
| Load-use on <b>rs2</b>    | $LD \rightarrow I+1$     | 1-cycle stall + ForwardB=01      | 1       |
| Store after ALU           | $ALU \rightarrow SD$     | ForwardB = 10                    | 0       |
| Store after load          | $LD \rightarrow SD$      | MEM-to-MEM forward (no stall)    | 0       |
| WB→ID same-cycle          | $I \rightarrow I+3$      | Register file write-first bypass | 0       |
| Branch correct prediction | —                        | Predicted PC is correct          | 0       |
| Branch misprediction      | —                        | Flush IF/ID, redirect PC         | 1       |

## 7 Verification and Testing

### 7.1 Test Methodology

Two verification testbenches exercise the processor:

1. `pipe_tb.v` — functional testbench that runs an arbitrary instruction stream and dumps the register file plus total cycle count.
2. `hazard_tb.v` — directed self-checking testbench with 26 individual register-value assertions covering every hazard class, including expected-value comparisons that report PASS/FAIL per test and an overall summary.

## 7.2 Directed Test Plan

Table 5: Directed test cases mapped to hazard scenarios.

| #  | Test Name          | Description                                      | Reg     | Expected       |
|----|--------------------|--------------------------------------------------|---------|----------------|
| 1  | EX haz. rs1        | add x4=x1+x2; add<br>x5=x4+x1                    | x5      | 20             |
| 2  | EX haz. rs2        | add x6=x1+x2; add<br>x7=x1+x6                    | x7      | 20             |
| 3  | MEM haz. rs1       | add x8=x1+x2; nop; add<br>x9=x8+x1               | x9      | 20             |
| 4  | MEM haz. rs2       | add x10=x1+x2; nop; add<br>x11=x1+x10            | x11     | 20             |
| 5  | Double hazard      | addi x12=1; addi x12+=2;<br>add x13=x12+x1       | x13     | 8              |
| 6  | Store to mem       | sd x1, 0(x0)                                     | —       | (mem verified) |
| 7  | Load-use rs1       | ld x14, 0(x0); add<br>x15=x14+x2                 | x15     | 15             |
| 8  | Load-use rs2       | ld x16, 0(x0); add<br>x17=x2+x16                 | x17     | 15             |
| 9  | Store after ALU    | addi x18=25; sd x18,<br>8(x0); ld x19, 8(x0)     | x19     | 25             |
| 10 | Store after load   | ld x20, 0(x0); sd x20,<br>16(x0); ld x21, 16(x0) | x21     | 5              |
| 11 | Branch not taken   | beq x1,x2,...; addi<br>x22=42                    | x22     | 42             |
| 12 | Branch taken       | beq x0,x0,+8; addi<br>x23=99; addi x24=77        | x23,x24 | 0, 77          |
| 13 | No hazard (3 NOPs) | addi x25=3; nop; nop; add<br>x26=x25+x1          | x26     | 8              |

## 7.3 Sample Test Walkthroughs

### 7.3.1 Test 1: EX-Stage Forwarding (ForwardA = 10)

```

1 addi x1, x0, 5      # x1 = 5
2 addi x2, x0, 10     # x2 = 10
3 add  x4, x1, x2      # x4 = 15
4 add  x5, x4, x1      # x5 = 20  <-- ForwardA=10 delivers x4

```

Listing 12: EX hazard test: back-to-back arithmetic.



Table 6: Pipeline diagram for Test 1 (EX forwarding).

| Cyc | IF    | ID    | EX       | MEM   | WB    | Forwarding |
|-----|-------|-------|----------|-------|-------|------------|
| 1   | addi1 |       |          |       |       |            |
| 2   | addi2 | addi1 |          |       |       |            |
| 3   | add4  | addi2 | addi1    |       |       |            |
| 4   | add5  | add4  | addi2    | addi1 |       |            |
| 5   |       | add5  | add4(15) | addi2 | addi1 |            |
| 6   |       |       | add5(20) | add4  | addi2 | FwdA=10    |

**Result:**  $x5 = 20$  ✓. The ALU result of add  $x4$  is forwarded from the EX/MEM pipeline register to the ALU's A input via ForwardA = 2'b10.

### 7.3.2 Test 7: Load-Use Hazard (Stall + Forward)

```

1 sd    x1, 0(x0)           # mem[0] = 5   (setup)
2 ld    x14, 0(x0)          # x14 = 5
3 add   x15, x14, x2         # STALL, then x15 = 5 + 10 = 15

```

Listing 13: Load-use hazard test: stall one cycle, then forward.

Table 7: Pipeline diagram for Test 7 (load-use stall).

| Cyc   | IF    | ID    | EX            | MEM           | WB | Stall?  |
|-------|-------|-------|---------------|---------------|----|---------|
| $k$   | add15 | ld14  | sd            |               |    | 0       |
| $k+1$ | add15 | ld14  | <i>bubble</i> | sd            |    | 1       |
| $k+2$ | ...   | add15 | ld14          | <i>bubble</i> | sd | 0       |
| $k+3$ |       | ...   | add15(15)     | ld14          |    | FwdA=01 |

**Result:**  $x15 = 15$  ✓. HDU inserts one bubble, then MEM/WB forwarding delivers the loaded value.

### 7.3.3 Test 10: Store After Load (MEM-to-MEM Forwarding)

```

1 ld    x20, 0(x0)          # x20 = mem[0] = 5
2 sd    x20, 16(x0)         # mem[16] = 5   (MEM-to-MEM forward)
3 ld    x21, 16(x0)         # x21 = mem[16] = 5   (verification)

```

Listing 14: Store-after-load: MEM-to-MEM forwarding avoids stall.

Table 8: Pipeline diagram for Test 10 (MEM-to-MEM forwarding, no stall).

| Cyc | IF   | ID   | EX   | MEM        | WB    |
|-----|------|------|------|------------|-------|
| 1   | ld20 |      |      |            |       |
| 2   | sd20 | ld20 |      |            |       |
| 3   | ld21 | sd20 | ld20 |            |       |
| 4   |      | ld21 | sd20 | ld20       |       |
| 5   |      |      | ld21 | sd20→M[16] | x20=5 |

**Result:**  $x_{21} = 5$  ✓. The `sd` in MEM receives the loaded value of  $x_{20}$  directly via MEM-to-MEM forwarding, requiring **zero stall cycles**.

### 7.3.4 Test 12: Branch Taken (Misprediction Flush)

```

1 beq  x0, x0, 8      # Always taken (x0 == x0)
2 addi x23, x0, 99    # SKIPPED (flushed from pipeline)
3 addi x24, x0, 77    # Branch target: x24 = 77

```

Listing 15: Branch taken: flush the wrongly-fetched instruction.

Table 9: Pipeline diagram for Test 12 (branch misprediction flush).

| Cyc | IF     | ID     | EX            | MEM | WB | Flush? |
|-----|--------|--------|---------------|-----|----|--------|
| 1   | beq    |        |               |     |    | 0      |
| 2   | addi23 | beq    |               |     |    | 0      |
| 3   | addi24 | addi23 | taken         |     |    | 1      |
| 4   | ...    | addi24 | <i>bubble</i> |     |    | 0      |

**Result:**  $x_{23} = 0$  (flushed),  $x_{24} = 77$  (branch target) ✓.

## 8 Simulation Results

Photos added in a separate folder

### 8.1 Forwarding Waveform Analysis

When two back-to-back R-type instructions exhibit a RAW dependency (e.g., `add x4,x1,x2` followed by `add x5,x4,x1`), the waveform shows that the value of `rd` in the EX/MEM stage matches `rs1` (or `rs2`) in the ID/EX stage. The `reg_write_en_EX_MEM` signal is asserted, satisfying the EX hazard conditions. GTKWave confirms that `ForwardA` and `ForwardB` transition to their appropriate values (2'b10 for EX-to-EX forwarding), and the ALU receives the correct operand in the same cycle—no stall is inserted.

### 8.2 Stall Waveform Analysis

For a load-use sequence (e.g., `ld x14,0(x0)` followed by `add x15,x14,x2`), the waveform shows `rd_ID_EX` matching the dependent `rs2` value in IF/ID while `mem_read` is asserted. The hazard detection unit raises `stall`, causing `pc_write` and `IF_ID_write` to be de-asserted for one cycle. The control signals are zeroed (bubble injection) and the instruction is stalled for exactly one additional cycle. After resuming, MEM/WB forwarding delivers the loaded value.

### 8.3 Flush Waveform Analysis

When a branch is taken (e.g., `beq x0,x0,+8`) and the prediction was “not taken,” the branch condition evaluates to true in the ID stage. The `flush` signal is asserted, and on the next clock edge the `instr_IF_ID` register is cleared to zero (NOP). The `pc_next` value

changes to the branch target address. The penalty for a mispredicted flush is **one clock cycle** (since branches are resolved in ID, only the single instruction already fetched in IF is squashed).

## 8.4 Stall Followed by Flush

A particularly interesting edge case occurs when a load-use stall is immediately followed by a branch misprediction. The waveform shows the stall occurring first (`instr_IF_ID` held, control signals zeroed), and in the immediately following cycle the flush is asserted, clearing the IF/ID register. The processor correctly handles this compound event: the stall completes, the flush squashes the misfetched instruction, and execution resumes from the correct target.

## 9 Module Summary

Table 10: Verilog modules and their roles.

| Module                             | Description                                                                                                         |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>pipelined_processor</code>   | Top-level 5-stage pipeline interconnecting all sub-modules, pipeline registers, forwarding muxes, and control logic |
| <code>forwarding_unit</code>       | Combinational detection of EX and MEM data hazards; outputs <code>ForwardA</code> , <code>ForwardB</code>           |
| <code>hazard_detection_unit</code> | Detects load-use hazards in ID; asserts <code>stall</code> (excludes store-after-load)                              |
| <code>branch_predictor</code>      | 64-entry BHT (2-bit counters) + 64-entry BTB; predicts taken/not-taken and target address                           |
| <code>register_file_module</code>  | 32×64-bit register file; write-first bypass for simultaneous WB write and ID read                                   |
| <code>program_counter</code>       | 64-bit PC register with synchronous update, halt, and stall control                                                 |
| <code>instruction_memory</code>    | 4096-byte ROM; loads hex instructions from file at simulation start                                                 |
| <code>control_unit</code>          | Decodes 7-bit opcode into 7 control signals                                                                         |
| <code>imm_gen</code>               | Extracts and sign-extends immediates for I, S, and B formats                                                        |
| <code>alu_control</code>           | Maps ( <code>alu_op</code> , <code>funct3</code> , <code>funct7[5]</code> ) to 4-bit ALU operation code             |
| <code>alu_64_bit</code>            | 64-bit ALU supporting 10 operations with flag outputs                                                               |
| <code>data_memory</code>           | 1024-byte synchronous RAM for loads and stores                                                                      |

## 10 Conclusion

This project demonstrates that a carefully designed hazard-resolution strategy can eliminate nearly all pipeline stall penalties:

- **Data forwarding** resolves the most common RAW hazards (EX and MEM) in zero cycles.

- **A single-cycle stall** suffices for load-use hazards, after which MEM/WB forwarding delivers the result.
- **MEM-to-MEM forwarding** is a targeted optimization that removes the stall from store-after-load sequences entirely.
- **Early branch resolution** (in ID) combined with a **2-bit dynamic predictor with BTB** confines the misprediction penalty to one cycle and eliminates the penalty altogether for correctly predicted branches.
- The **write-first register file bypass** closes the last remaining forwarding gap (WB writing and ID reading the same register in the same cycle).

Across 26 directed test scenarios—covering every forwarding path, both stall trigger conditions, double data hazards, store optimizations, and branch control flow—the processor produces correct results with the expected cycle count. The sequential-vs.-pipelined comparison confirms functional equivalence (identical register contents), while the pipelined design unlocks a significantly higher clock frequency, validating the completeness and correctness of the implementation.

## 11 Contribution

This project was completed through close collaboration among all three team members. While responsibilities were distributed to ensure efficient development, the design, debugging, and testing of the processor were carried out through continuous discussions and joint problem solving. The contributions of each member are summarized below.

- **Saumya**
  - Led the overall **processor architecture design** and coordination of the datapath components.
  - Implemented the **Program Counter (PC)** and contributed significantly to the **Control Unit**.
  - Designed and integrated the **pipeline registers** required for the pipelined processor.
  - Contributed to the **ALU integration** and debugging of datapath connections.
  - Participated actively in **testing, verification, and debugging** of the sequential and pipelined implementations.
- **Het**
  - Implemented the **Instruction Memory interface** and file-based instruction loading.
  - Designed the **Register File** and contributed to the **Immediate Generation** unit.
  - Implemented the **Forwarding Unit** used for resolving data hazards in the pipeline.

- Assisted in **integration of datapath modules** and debugging of pipeline execution.
- Contributed to **test case development** and validation of processor functionality.
- **Shrenil**
  - Implemented the **ALU Control Unit** and contributed to the integration of the **ALU** in the processor datapath.
  - Designed the **Data Memory** module and its interaction with load/store instructions.
  - Contributed to the **hazard detection logic** and pipeline integration.
  - Assisted in **debugging pipeline behavior** and validating instruction execution.
  - Participated in **testing and documentation** of the processor.

In addition to the individual responsibilities, several tasks were carried out collaboratively. The **hazard detection unit**, **load-store forwarding support**, project **documentation**, and the preparation of this report were completed jointly by all three members.

## References

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed., Morgan Kaufmann, 2021.